

Lab6 实验报告

一、思考题

1、父进程为读者

- 示例代码中，父进程操作管道的写端，子进程操作管道的读端。如果现在想让父进程作为“读者”，代码应当如何修改？

```
#include <stdlib.h>
#include <unistd.h>
int fildes[2];
char buf[100];
int status;
int main(){
    status = pipe(fildes);

    if(status == -1) {
        printf("error\n");
    }

    switch(fork()) {
        case -1:
            break;
        case 0: /*子进程-作为管道的写者*/
            close(fildes[0]); /*关闭不用的读端*/
            write(fildes[1], "Hello world\n", 12); /*向管道中写数据*/
            close(fildes[1]); /*写入结束，关闭写端*/
            exit(EXIT_SUCCESS);
        default: /*父进程-作为管道的读者*/
            close(fildes[1]); /*关闭不用的写端*/
            read(fildes[0], buf, 100); /*从管道中读数据*/
            printf("father-process read:%s", buf); /*打印读到的数据*/
            close(fildes[0]); /*读取结束，关闭读端*/
            exit(EXIT_SUCCESS);
    }
}
```

2、dup 中的进程竞争

- 上面这种不同步修改 pp_ref 而导致的进程竞争问题在 user/lib/fd.c 中的dup 函数中也存在。请结合代码模仿上述情景，分析一下我们的 dup函数中为什么会出现预想之外的情况？

```
int dup(int oldfdnum, int newfdnum) {
    // todo

    /* Step 2: Close 'newfdnum' to reset content. */
    close(newfdnum);
    /* Step 3: Get 'newfd' reffered by 'newfdnum'. */
    newfd = (struct Fd *)INDEX2FD(newfdnum);
    /* Step 4: Get data address. */
```

```

ova = fd2data(oldfd);
nva = fd2data(newfd);
/* Step 5: Duplicate the data and 'fd' self from old to new. */

if ((r = syscall_mem_map(0, oldfd, 0, newfd, vpt[VPN(oldfd)] & (PTE_D |
PTE_LIBRARY))) <
    0) {
    goto err;
}

if (vpd[PDX(ova)]) {
    for (i = 0; i < PDMAP; i += PTMAP) {
        pte = vpt[VPN(ova + i)];

        if (pte & PTE_V) {
            // should be no error here -- pd is already allocated
            if ((r = syscall_mem_map(0, (void *) (ova + i), 0, (void *) (nva +
i),
                                pte & (PTE_D | PTE_LIBRARY))) < 0) {
                goto err;
            }
        }
    }
}

return newfdnum;

err:
    // todo
}

```

`dup` 函数的作业是把一个文件描述块 `oldfdnum` 复制到 `newfdnum`，可以发现 `dup` 函数中也是先映射文件描述块，再映射具体的文件描述块的内容。考虑下面一段代码。

```

pipe(p);
if (fork() == 0) {
    read(p[0], buf, sizeof(buf));
} else {
    dup(p[0], newfd);
    write(p[1], "Hello", 5);
}

```

- 父进程在 `dup(p[0], newfd)` 过程中，完成了 `newfd` 对 `p[0]` 的映射，此时 `pageref(p[0]) = pageref(pipe) = 2`。
- 子进程执行 `read`，`read` 中首先判断写者是否关闭。比较 `pageref(pipe)` 与 `pageref(p[0])` 之后发现它们都是 2，说明写端已经关闭，于是子进程退出。
- 此时父进程永远无法结束

3、原子操作

- 阅读上述材料并思考：为什么系统调用一定是原子操作呢？如果你觉得不是所有的系统调用都是原子操作，请给出反例。希望能结合相关代码进行分析说明。

如果系统调用不是原子性的，可能再执行系统调用的过程中发生中断，此时可能会出现预料之外的结果导致出错。

4、解决进程竞争

- 仔细阅读上面这段话，并思考下列问题

1. 按照上述说法控制 pipe_close 中 fd 和 pipeunmap 的顺序，是否可以解决上述场景的进程竞争问题？给出你的分析过程。

显然可以解决。有 $\text{pageref}(\text{pipe}) \geq \text{pageref}(\text{fd})$ ，先取消 fd 的映射会减小 $\text{pageref}(\text{fd})$ ，此时只会有 $\text{pageref}(\text{pipe}) > \text{pageref}(\text{fd})$ 而不会出现二者相等的情况，因此不会误判另一端已经关闭。

2. 我们只分析了 close 时的情形，在 fd.c 中有一个 dup 函数，用于复制文件描述符。试想，如果要复制的文件描述符指向一个管道，那么是否会出现与 close 类似的问题？请模仿上述材料写写你的理解。

可能出现这种情况，详细分析见上一道思考题。

5、如何实现加载 bss 段

- 思考以下三个问题。

1. 认真回看 Lab5 文件系统相关代码，弄清打开文件的过程。
 2. 回顾 Lab1 与 Lab3，思考如何读取并加载 ELF 文件。
 3. 在 Lab1 中我们介绍了 data text bss 段及它们的含义，data 段存放初始化过全局变量，bss 段存放未初始化的全局变量。关于 memsize 和 filesize，我们在 Note1.3.4 中也解释了它们的含义与特点。关于 Note1.3.4，注意其中关于“bss 段并不在文件中占数据”表述的含义。回顾 Lab3 并思考：`elf_load_seg()` 和 `load_icode_mapper()` 函数是如何确保加载 ELF 文件时，bss 段数据被正确加载进虚拟内存空间。bss 段在 ELF 中并不占空间，但 ELF 加载进内存后，bss 段的数据占据了空间，并且初始值都是 0。请回顾 `elf_load_seg()` 和 `load_icode_mapper()` 的实现，思考这一点是如何实现的？
- 下面给出一些对于上述问题的提示，以便大家更好地把握加载内核进程和加载用户进程的区别与联系，类比完成 `spawn` 函数。

关于第一个问题，在 Lab3 中我们创建进程，并且通过 `ENV_CREATE(...)` 在内核态加载了初始进程，而我们的 `spawn` 函数则是通过和文件系统交互，取得文件描述块，进而找到 ELF 在“硬盘”中的位置，进而读取。关于第二个问题，各位已经在 Lab3 中填写了 `load_icode` 函数，实现了 ELF 可执行文件中读取数据并加载到内存空间，其中通过调用 `elf_load_seg` 函数来加载各个程序段。在 Lab3 中我们要填写 `load_icode_mapper` 回调函数，在内核态下加载 ELF 数据到内存空间；相应地，在 Lab6 中 `spawn` 函数也需要在用户态下使用系统调用为 ELF 数据分配空间。

1. 用户打开文件就是调用 `open()` 函数，再 `open()` 函数中调用 `fsipc_open()` 给文件服务进程发送消息，文件服务进程接收到消息后调用 `serve_open()` 函数，然后创建文件并打开，将 `FileFd` 传递回去。
2. 读取 ELF 文件，只需要知道结构体 `struct Elf32_Ehdr` 的结构，然后依次读取段头表和节头表即可。加载 ELF 文件在 `kern/env.c` 中的 `load_icode` 中实现。
3. 在 `elf_load_seg()` 中，如果 `bin_size < sgsz` 时会调用回调函数 `elf_mapper_t`。这里的回调函数就是 `load_icode_mapper()`，该函数会再次进行 `page_insert`，保证申请了足够的地址空间来存放 bss 区域，同时通过设置 `src = NULL` 使初始值都是 0。

6、内置命令与外部命令

- 通过阅读代码空白段的注释我们知道，将标准输入或输出定向到文件，需要我们将 `dup` 到 0 或 1 号文件描述符 (fd)。那么问题来了：在哪步，0 和 1 被“安排”为标准输入和标准输出？请分析代码执行流程，给出答案。

```
// use/init.c
// todo
int main(int argc, char **argv) {
    //todo
    // stdin should be 0, because no file descriptors are open yet
    if ((r = opencons()) != 0) {
        user_panic("opencons: %d", r);
    }
    // stdout
    if ((r = dup(0, 1)) < 0) {
        user_panic("dup: %d", r);
    }

    // todo
}
```

在此处，0和1被设置为标准输入和标准输出。

7、标准输入和标准输出

- 在 shell 中执行的命令分为内置命令和外部命令。在执行内置命令时 shell 不需要 fork 一个子 shell，如 Linux 系统中的 cd 命令。在执行外部命令时 shell 需要 fork 一个子 shell，然后子 shell 去执行这条命令。据此判断，在 MOS 中我们用到的 shell 命令是内置命令还是外部命令？请思考为什么 Linux 的 cd 命令是内部命令而不是外部命令？

1. shell命令是外部命令，在sh.c中定义。
2. 因为 cd 需要修改当前 shell 进程的工作目录，这必须在当前进程内部完成。如果把 cd 设计为外部命令，shell 会依次fork() 一个子进程；在子进程中执行cd指令跳到对应目录；然后子进程退出，shell 回到原来的目录。可以发现，父 shell 的工作目录没有变，因为子进程的任何工作目录改变都无法影响到父进程的工作目录。

8、解释命令执行的现象

- 在你的 shell 中输入命令 ls.b | cat.b > motd。

```
$ ls.b | cat.b > motd
[00002803] pipecreate
[00003805] destroying 00003805
[00003805] free env 00003805
i am killed ...
[00004006] destroying 00004006
[00004006] free env 00004006
i am killed ...
[00003004] destroying 00003004
[00003004] free env 00003004
i am killed ...
[00002803] destroying 00002803
[00002803] free env 00002803
i am killed ...
```

1. 请问你可以在你的shell 中观察到几次spawn? 分别对应哪个进程?

```
$ ls.b | cat.b > motd
[00002803] pipecreate
spawn: father 2803, child 3805
spawn: father 3004, child 4006
[00003805] destroying 00003805
[00003805] free env 00003805
i am killed ...
[00004006] destroying 00004006
[00004006] free env 00004006
i am killed ...
[00003004] destroying 00003004
[00003004] free env 00003004
i am killed ...
[00002803] destroying 00002803
[00002803] free env 00002803
i am killed ...
```

在spawn.c中加入语句 `debugf("spawn: father %x, child %x\n", syscall_getenv(), child);`，可以看到调用了两次spawn，分别是进程2803生成了3805，和进程3004生成了4006。

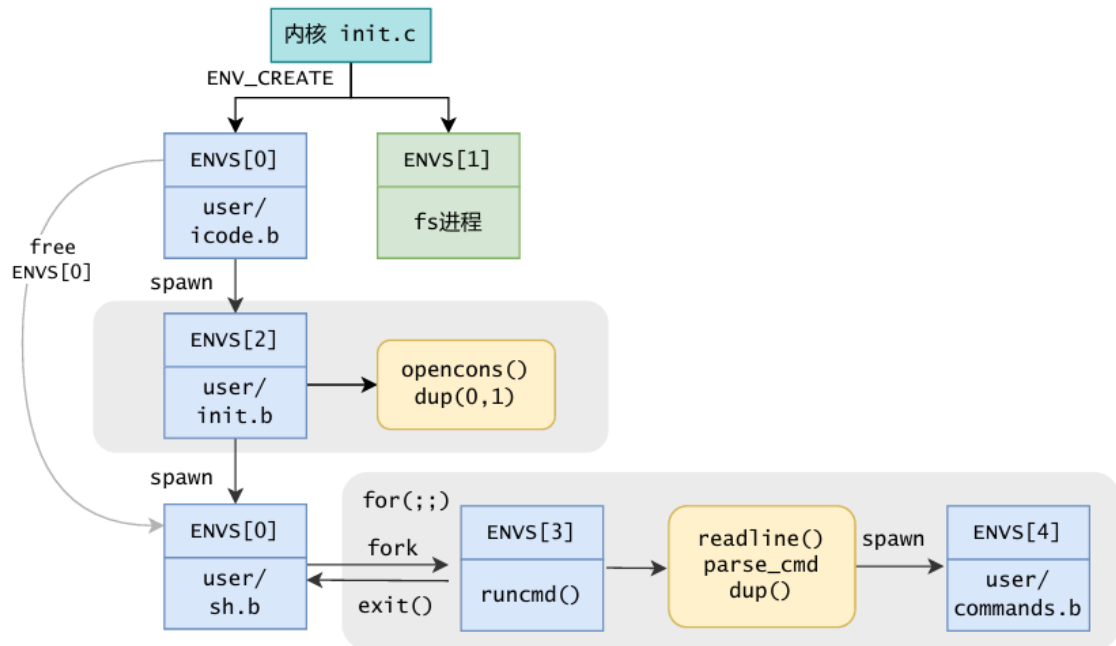
2. 请问你可以在你的 shell 中观察到几次进程销毁？分别对应哪个进程？

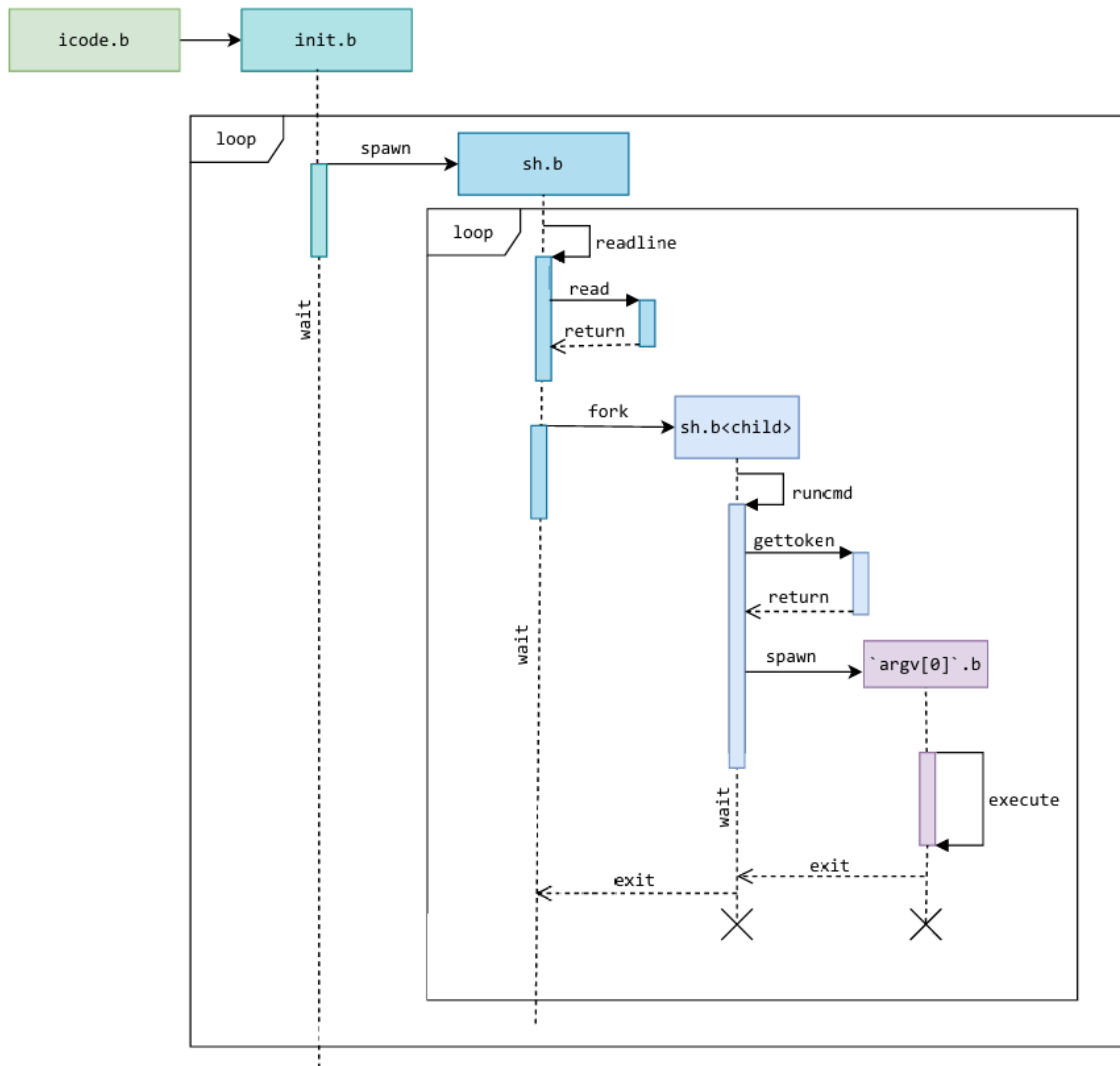
读图可看到四次进程销毁。i am killed ...

二、难点分析

本次实验我觉得第一个难点就是理解进程同步与互斥的安全问题，，因为管道本身的共享性质，在当前这种不加锁控制的情况下，无法保证 `_pipe_is_closed`用于管道另一端关闭的判断一定返回正确的结果。解决方法也很简单显然有 `pageref(pipe) >= pageref(fd)`，而如果先取消pipe的映射，可能会导致 `pageref(pipe) > pageref(fd)`变成 `pageref(pipe) = pageref(fd)`，但是先取消fd的映射就不会出现这个情况，因此不会误判另一端已经关闭。

还有个进程安全问题是读取 pp_ref 的同步问题，fd 是一个父子进程共享的变量，但子进程中的 pageref(fd) 没有随父进程对 fd 的修改而同步，这就造成了子进程读到的 pageref(fd) 成为了“脏数据”。为了保证读的同步性，子进程应当重新读取 pageref(fd) 和 pageref(pipe)，并且要在确认两次读取之间进程没有切换后，才能返回正确的结果。使用 env_runs 即可解决这个问题。虽然这些同步问题解决起来都很简单，但是如果让我自己设计一定想不到这些东西，还是对操作系统理解不够深入，对于同步互斥不够敏感。





第二个难点就是实现一个简易的shell模式。关键就是理解shell的启动过程和相关函数的调用关系，具体函数的实现相比起来就比较简单。

三、实验体会

终于到了最后一次实验，因为这次实验不需要上机考试，所以做起来也有点敷衍，很多东西都没有搞清楚，没有整体地去思考这个Lab地内容，而且又是两周前写的实验，导致在写实验报告地生活有种盲人摸象地感觉，从头看了遍指导书才理解了很多当初做实验时没理解的内容。最后就祝北航操作系统课程越做越好，也感谢课程组老师和助教一个学期的辛勤劳作！

四、原创说明

- 本文部分内容在LLM（大语言模型）的协助下完成